

Dmitry Muravyev

Delta encoding

Sprint 4: Mathematical foundation

Notation

Symbol	Meaning
N	number of frames (pixels, samples)
C	number of channels
$x_{c,i}$	value of channel c in frame i, $0 \le i < N$
k_c	bits per channel c (target bit depth)
$F = \sum_c k_c$	bits per frame
b	block size field (bits); block length $1 \le l \le 2^b$
$m,\ w$	method and window width (bits)
$M_c,\ d_c$	number of methods and bits per method declaration
$H = b + \sum_c d_c$	header size of a block
$L_{c,m}(p)$	longest block from frame p for which method m of channel c works
$\mathrm{Bits}(p, l)$	size of the cheapest block of length l starting at frame p
$\mathrm{Best}(p)$	size of the cheapest chain of blocks covering frames p … N−1

All sizes are in bits.

Frames and squares

- A frame is the set of all channel values of one pixel (sample):
  - $x_i = (x_{0,i},\ x_{1,i},\ \ldots,\ x_{C-1,i})$, e.g. RGB 565: $k = (5, 6, 5),\ F = 16$

- Splitting into s × s squares (s divides the image width W and height):
  - frame i is taken from pixel π(i):

$$q = \lfloor i/s^2 \rfloor, \quad o = i \bmod s^2, \quad r = \lfloor q/(W/s) \rfloor, \quad u = q \bmod (W/s)$$

$$\pi(i) = (r \cdot s + \lfloor o/s \rfloor) \cdot W + u \cdot s + (o \bmod s)$$

The squares keep neighboring pixels close in the frame sequence, so the deltas are smaller.

s = 1 means no splitting: π(i) = i.

0	1	4	5	8	9	12	13
2	3	6	7	10	11	14	15
16	17	20	21	24	25	28	29
18	19	22	23	26	27	30	31

Frame numbers for 2 × 2 squares, W = 8

Block format

Block frames l − 1	Method Channel 0	...	Method Channel C−1	Initial value Channel 0	...	Initial value Channel C−1	Data ...
b bits	d_0 bits		d_{C−1}	k_0 bits*		k_{C−1} bits*	

** the initial value is omitted for an uncompressed channel (m = k).*

$$d_c = \lfloor \log_2 (M_c - 1) \rfloor + 1, \qquad M_c = \{k_c + 1 \text{ (single } \Delta \text{ option)}, \quad 2k_c \text{ (two } \Delta \text{ options)}\}$$

$$H = b + \sum_{c=0}^{C-1} d_c$$

Channel bits k	2	3	4	5	6	7	8
d, single Δ option	2	2	3	3	3	3	4
d, two Δ options	2	3	3	4	4	4	4

Block length is stored as l − 1,
so b bits give blocks of 1 … 2^b frames.
Every block is self-contained: the decoder needs no state from the previous block.

Methods

m	Fixed window only	Adaptive window only	Fixed window first	Adaptive window first
0	Fixed, w = 0	Fixed, w = 0	Fixed, w = 0	Fixed, w = 0
1 … k−1	Fixed, w = m	Adaptive, w = m	Fixed, w = m	Adaptive, w = m
k	Uncompressed	Uncompressed	Uncompressed	Uncompressed
k+1 … 2k−1	—	—	Adaptive, w = m − k	Fixed, w = m − k

Cost of a channel in a block of l frames (bits, without the header):

Fixed window: $\text{cost} = k + l \cdot w$ (initial value = minimum of the block)

Adaptive floating window: $\text{cost} = k + (l - 1) \cdot w$ (initial value = first frame)

Uncompressed: $\text{cost} = l \cdot k$

All frames equal (Fixed, w = 0): $\text{cost} = k$

The Δ option is a property of the whole stream: the compressor and the decoder (#define in Decompression.h) must use the same one.

Fixed window

Method with window width w is applicable to frames p … p+l−1 if

$$\max_{p \le i < p+l} x_i - \min_{p \le i < p+l} x_i \le 2^w - 1$$

Initial value v = min, data x_i − v in w bits.

$L_w^F(p) = \max \{\, l \le \min(2^b,\ N - p) \ :$ range of frames $p \ldots p + l - 1 \ \le 2^w - 1 \,\}$

	100	110	120	110	115	120	130	140	150	160	170	180	190	200	210	220
100	0	10	20	10	15	20	30	40	50	60	70	80	90	100	110	120

w = 5 (0 ~ 31), block length: 7

	103	114	118	107	102	131	119	128	127	115	110	130	100	101	122	125
100	3	14	18	7	2	31	19	28	27	15	10	30	0	1	22	25

w = 5, block length: 16

Example cost (k = 8, w = 5, l = 16): 8 + 16 × 5 = 88 bits instead of 128 bits.

Adaptive floating window

Window of width 2^w around the previous frame, clamped to the value range 0 … 2^k − 1:

$$s_w(y) = \min\left(\max\left(y - 2^{w-1},\ 0\right),\ 2^k - 2^w\right)$$

Frame x_i fits if $s_w(x_{i-1}) \le x_i < s_w(x_{i-1}) + 2^w$, data $x_i - s_w(x_{i-1})$ in w bits.

$L_w^A(p) = \max\{\, l \le \min(2^b,\ N - p) \ :$ every frame $p+1 \ldots p+l-1$ fits $\}$, initial value $= x_p$

	100	110	120	110	115	120	130	140	150	160	170	180	190	200	210	220
	100	26	26	6	21	21	26	26	26	26	26	26	26	26	26	26
s:		84	94	104	94	99	104	114	124	134	144	154	164	174	184	194

w = 5, block length: 16. Cost: 8 + 15 × 5 = 83 bits.

The window follows the source data (the previous frame is the original value, the compression is lossless), so the position of the window does not depend on w. This is what makes the windows nested (Lemma 1). Clamping keeps the whole window inside the value range, so no window bits are wasted at 0 and 2^k − 1.

Block size and efficiency

$$\text{Bits}(B) = H + \sum_{c=0}^{C-1} \text{cost}_c(m_c,\ l)$$

Method m_c may be used only if $l \le L_{c,m_c}(p)$.

$$E_{block} = \left(1 - \frac{\text{Bits}(B)}{l \cdot F}\right) \times 100\%, \qquad E_{overall} = \left(1 - \frac{\sum_j \text{Bits}(B_j)}{N \cdot F}\right) \times 100\%$$

The output file is the chain of blocks B_0, B_1, … covering all N frames, padded to whole bytes:

$$\text{bytes} = \left\lceil \frac{1}{8} \sum_j \text{Bits}(B_j) \right\rceil$$

Max eff for k = 8: the data part of a block can't be smaller than w/k of the raw data.

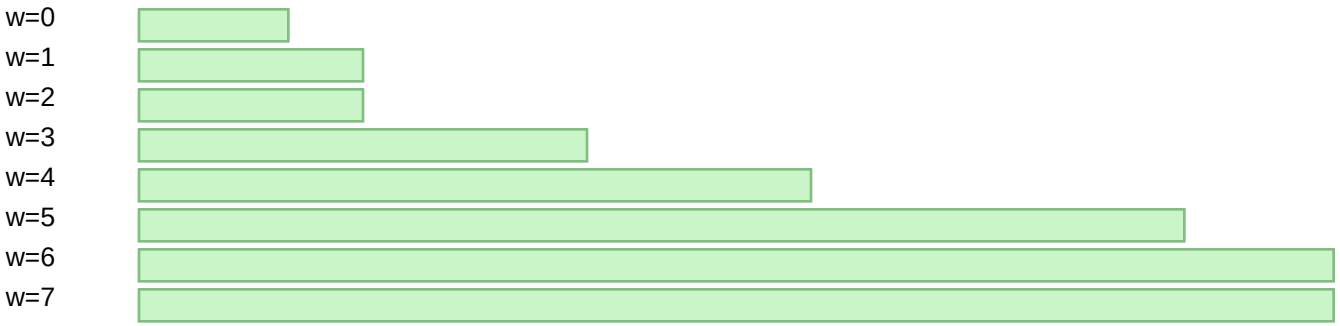
Window bits	Max eff
0	100.0%
1	87.5%
2	75.0%
3	62.5%
4	50.0%
5	37.5%
6	25.0%
7	12.5%
8	0.0%

Lemma 1: nested windows

For every start frame p and channel:

$$L_0^F(p) \le L_1^F(p) \le \ldots \le L_{k-1}^F(p), \qquad L_1^A(p) \le L_2^A(p) \le \ldots \le L_{k-1}^A(p)$$

- Fixed: the condition range ≤ 2^w − 1 only gets weaker as w grows.
- Adaptive: for the same previous frame y the window of width 2^(w+1) contains the window of width 2^w:
  - $s_{w+1}(y) \le s_w(y)$ and $s_w(y) + 2^w \le s_{w+1}(y) + 2^{w+1}$
  - without clamping: y − 2^w ≤ y − 2^(w−1) and y + 2^(w−1) ≤ y + 2^w;
  - clamping at 0 or 2^k − 2^w moves both windows the same way (case analysis,
  - also checked exhaustively for all y, w and k ≤ 12).
- The previous frame is the source value, the same for all w, so the nesting holds frame by frame.



Consequence: one scan of the data finds the lengths of all windows at once (the break points of w = 0, 1, 2, … in turn).
O(2^b + k) instead of O(k · 2^b) per channel.

Lemma 2: prefixes and the cheapest method

- If a method works for l frames from p, it works for any shorter block from p:
  - Fixed: a prefix has a smaller (or equal) range; Adaptive: a prefix has fewer frames to fit.
- So for a block of exactly l frames, channel c may use every method with $L_{c,m}(p) \geq l$.
- For the same l, a wider window only costs more, so the best choice in each family is the
  - narrowest window that lasts long enough:

$$w^F(l) = \min\{w : \ L_w^F(p) \geq l\}, \qquad w^A(l) = \min\{w : \ L_w^A(p) \geq l\}$$

$$c_c^*(p,l) = \min\left(l \cdot k, \quad k + l \cdot w^F(l), \quad k + (l-1) \cdot w^A(l)\right)$$

$$\text{Bits}(p,l) = H + \sum_{c=0}^{C-1} c_c^*(p,l)$$

Bits(p, l) is the exact size of the cheapest block of l frames starting at p: every decodable block of that length is at least this big, and the narrowest windows give a real block of exactly this size.

Finding the best common block: exact search

Sprints 2–3: a block is a combination of methods (m_0, …, m_{C−1}), its length is l = min_c L_{c,m_c}(p), and the best block has the highest E_block. Brute force tries all ∏ M_c combinations.

Group the combinations by their length. For a candidate length l (one of the lengths L_{c,m}):

$$A_c(l) = \min_{m:\, L_{c,m} \geq l} \text{cost}_c(m, l), \qquad B_c(l) = \min_{m:\, L_{c,m} = l} \text{cost}_c(m, l)$$

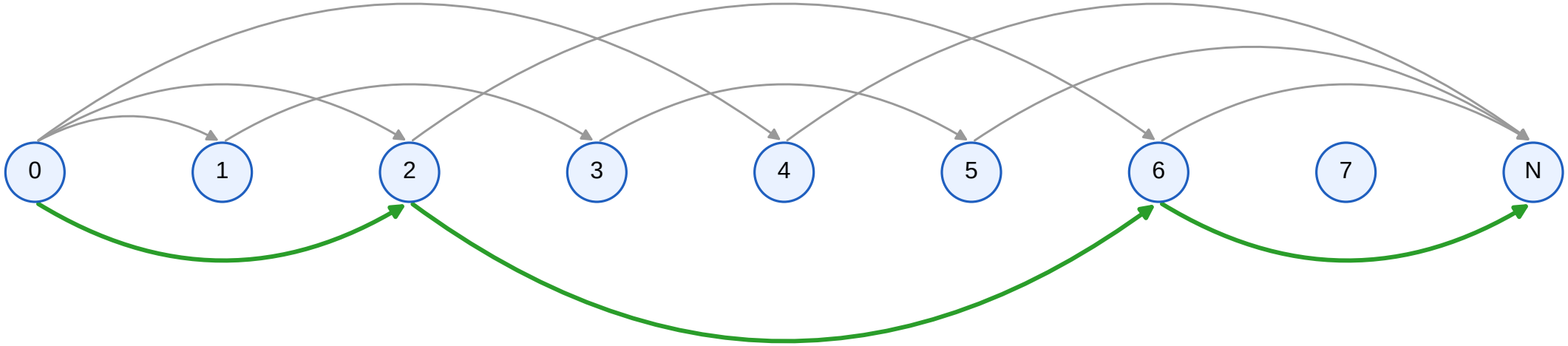
$$\text{Bits}^*(l) = H + \sum_c A_c(l) + \min_c \left(B_c(l) - A_c(l)\right)$$

- Proof: the length is exactly l if every channel lasts ≥ l and at least one channel j ends at l.
  - For a fixed j the best is B_j + Σ_{c≠j} A_c; minimizing over j gives the formula.
- Best block: $\max_l \; 1 - \text{Bits}^*(l) / (l \cdot F)$ over at most $\sum_c M_c$ candidate lengths.
- Ties: the lexicographically first (m_0, …, m_{C−1}), rebuilt channel by channel, so the result is the same
  - block as the brute force, byte for byte.

Complexity: O(Σ M_c · |candidates|) ≤ O((Σ M_c)²)
instead of O(∏ M_c).
Logo RGB 888: 17.5 s → 0.19 s, same output.

Frame format	Brute force, 2 Δ	Exact, 2 Δ (≤)
RGB 565	1 200	34^2 = 1 156
RGB 888	4 096	48^2 = 2 304
RGB 48 bit	32 768	96^2 = 9 216
4ch × 32 bit	16 777 216	256^2 = 65 536

Chain of blocks as a shortest path



Frame boundaries are nodes 0 … N. A block of l frames from p is an edge p → p + l with weight Bits(p, l).

Every compressed stream is a path from 0 to N, and its size is the path weight.

$$\text{Best}(N) = 0$$

$$\text{Best}(p) = \min_{1 \le l \le \min(2^b,\ N-p)} (\text{Bits}(p, l) + \text{Best}(p + l))$$

$$\text{size}_{min} = \text{Best}(0)$$

Best(p) is computed from p = N − 1 down to 0; the chosen length of every position is stored,
and the chain is restored from frame 0 forward.

Theorem: the chain is optimal

For the given block size field b, Δ option and squares, the restored chain has the minimum size among all streams that the decoder accepts.

Proof

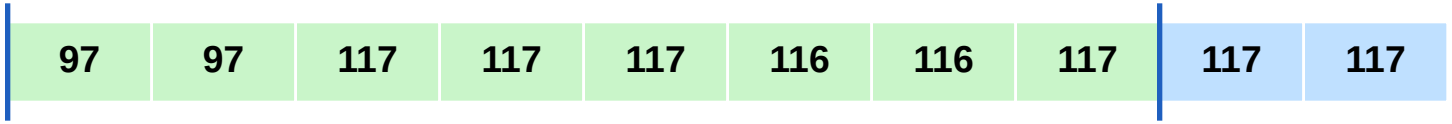
- Any accepted stream is a sequence of blocks with lengths $1 \le l_j \le 2^b$ and $\Sigma l_j = N$.
- In each block every channel uses an applicable method, so the block is at least Bits(p, l) (Lemma 2).
- By induction from p = N down to 0, a stream for frames p … N−1 is at least Best(p).
- The restored chain uses, in every block, the narrowest windows that reach Bits(p, l), with the initial
  - value = minimum (Fixed) or first frame (Adaptive), so its size is exactly Best(0). ■

Optimal "per block" (sprints 2–3: the most efficient block at each step, or the best block for every frame) is not optimal for the chain: a very efficient long block can leave a tail that compresses badly, and a short, less efficient block can open the way for a much better one. The example is on the next slide. Bits(p, l) does not depend on the previous blocks, so the dynamic programming covers all chains.

Example: greedy vs optimal chain

k = 8, b = 3 (blocks of 1 … 8 frames), Fixed window first (d = 4), H = 7 bits, raw data 80 bits.

Greedy: the most efficient block at each step

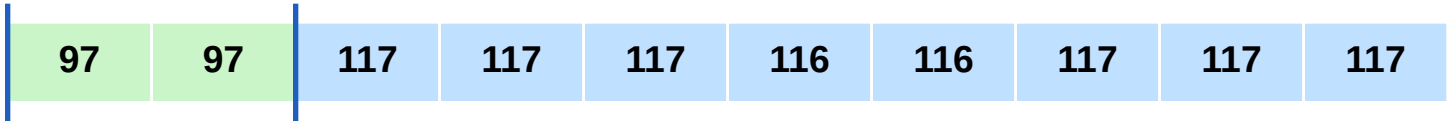


70 bits
E = 12.5%

Fixed w=5: 55 bits, E=14.06%

Fixed w=0: 15 bits, E=6.25%

Optimal chain (dynamic programming)



38 bits
E = 52.5%

Fixed w=0: 15 bits, E=6.25%

Fixed w=1: 23 bits, E=64.06%

Greedy: 7 + 8 + 8·5 = 55 (E = 14.1%) then 7 + 8 = 15 → 70 bits.

Optimal: 7 + 8 = 15 (E = 6.25%) then 7 + 8 + 8·1 = 23 (E = 64.1%) → 38 bits, 46% smaller.

Bits(p, l) is piecewise linear

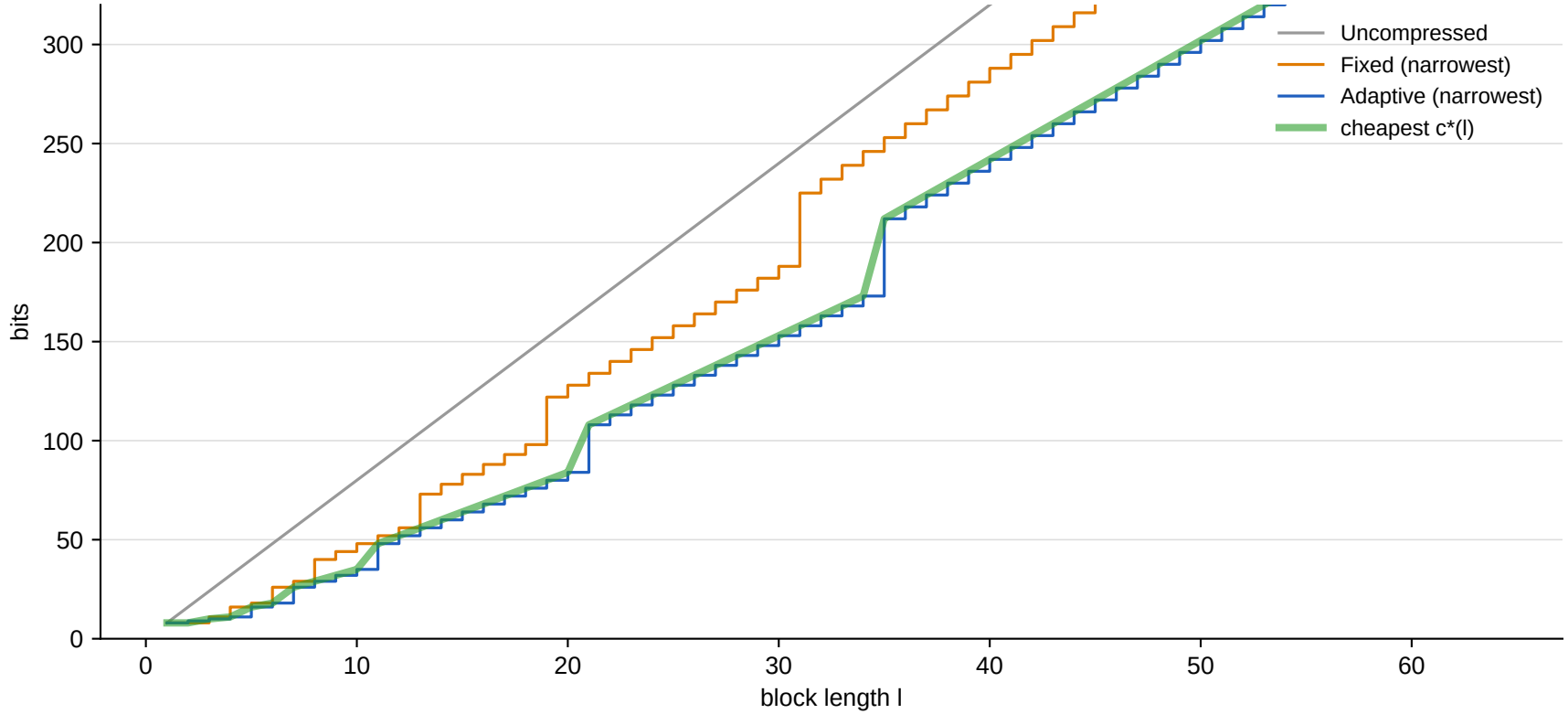
For one channel the candidates are three lines (with the narrowest windows that still last l frames):

$$\text{Uncompressed: } k \cdot l, \quad \text{Fixed: } k + w \cdot l, \quad \text{Adaptive: } (k - w) + w \cdot l$$

- w^F(l) and w^A(l) change only when l passes some L_w; between those points
  - the cheapest line a + s·l stays the cheapest until a line with a smaller slope crosses it:

$$l_{\times} = \left\lfloor \frac{a_i - a}{s - s_i} \right\rfloor + 1$$

So Bits(p, l) = α + σ·l on segments, and the next segment start is the nearest l_× or L_w + 1 over all channels. Inside a segment each length costs one addition.



Lemma 3: pruning

$c_c^*(p, l)$ never decreases as l grows:

- every candidate line has a non-negative slope and the windows only get wider;
- the set of applicable methods only shrinks (Lemma 2).

So $\mathrm{Bits}(p, l)$ is non-decreasing in l, and with $\mathrm{Best}(\cdot) \geq 0$:

$$\mathrm{Bits}(p, l) \geq \text{best so far} \;\Rightarrow\; \mathrm{Bits}(p, l') + \mathrm{Best}(p + l') \geq \text{best so far} \quad \forall\, l' \geq l$$

The scan of lengths stops at the first such l. No candidate is lost, so the result stays optimal.

In practice a block that is already bigger than the best whole tail appears after a few dozen frames,
so the scan looks at a small part of the 2^b possible lengths. Example: mit-text, b = 15 (32 768 lengths),
takes about 1 second for the whole file.

Incremental lengths (backward pass)

The DP goes from p = N − 1 down to 0, so the lengths L_w(p) are updated from L_w(p+1):

Fixed window

- End of the longest block $e_w(p) = p + L_w^F(p)$. If $[p, e)$ fits, $[p+1, e)$ fits too, so $e_w(p) \le e_w(p+1)$.
- Start from e = e_w(p+1) and move it left while e > p + 2^b or the range of [p, e) > 2^w − 1.
- The min and max of the window are kept in two monotonic deques (amortized O(1)).

Adaptive floating window

- A block breaks at the first frame that doesn't fit the window around its predecessor:

$$\beta_w(p) = p + 1 \text{ if } x_{p+1} \text{ doesn't fit around } x_p, \text{ otherwise } \beta_w(p) = \beta_w(p+1)$$

$$L_w^A(p) = \min\left(\beta_w(p) - p,\ 2^b,\ N - p\right)$$

All lengths of all windows for all positions: O(N · Σ_c k_c) amortized,
instead of rescanning up to 2^b frames for each window at each position: O(N · 2^b · Σ_c k_c).

Complexity

Step	Before (sprint 3)	Now
Lengths of all methods at p	O(Σ M_c · 2^b)	O(Σ k_c) amortized
Best common block at p	O(∏ M_c)	O((Σ M_c)²)
Chain of blocks	heuristic (frame-by-frame)	exact DP, pruned
Memory allocated, Logo RGB 888	41 GB (total)	< 30 MB
Compression time, all test cases	53.7 s	2.9 s
Decoder (flash / RAM)	—	unchanged

Compressor implementation: data cached once as arrays, Span<T> and stackalloc in the hot loops, no allocations per frame, multithreaded search for the legacy chain and for Auto settings.

Auto settings

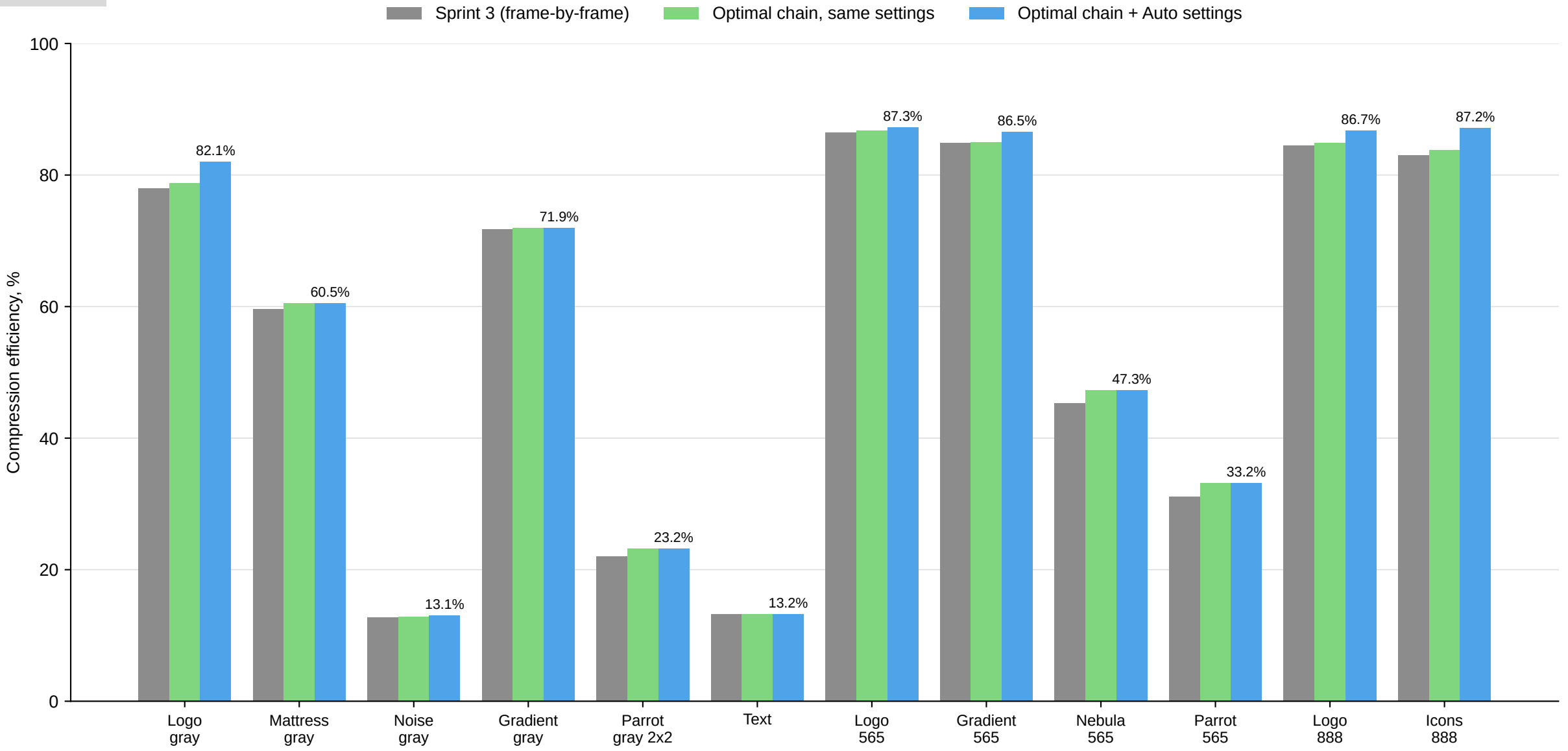
The DP gives the exact output size without writing the stream, so the settings can be searched:

$$(s^*,\ \delta^*,\ b^*) = \arg \min_{s \in S,\ \delta \in \Delta,\ b \in \mathcal{B}} \mathrm{Best}_{s,\delta,b}(0)$$

- S: square widths that divide the image width and height (1 = no squares);
- Δ: the 4 Δ options; $\mathcal{B}$: the block size field options.
- All (s, δ) pairs are tried in parallel: squares and Δ option interact,
  - choosing them one after another was up to 36% worse.
- For each pair b grows until the size gets worse 3 times in a row (in practice the
  - size has a single minimum in b; this stop rule is the only heuristic step).
- The predicted size equals the real output, so the choice is not an estimate.

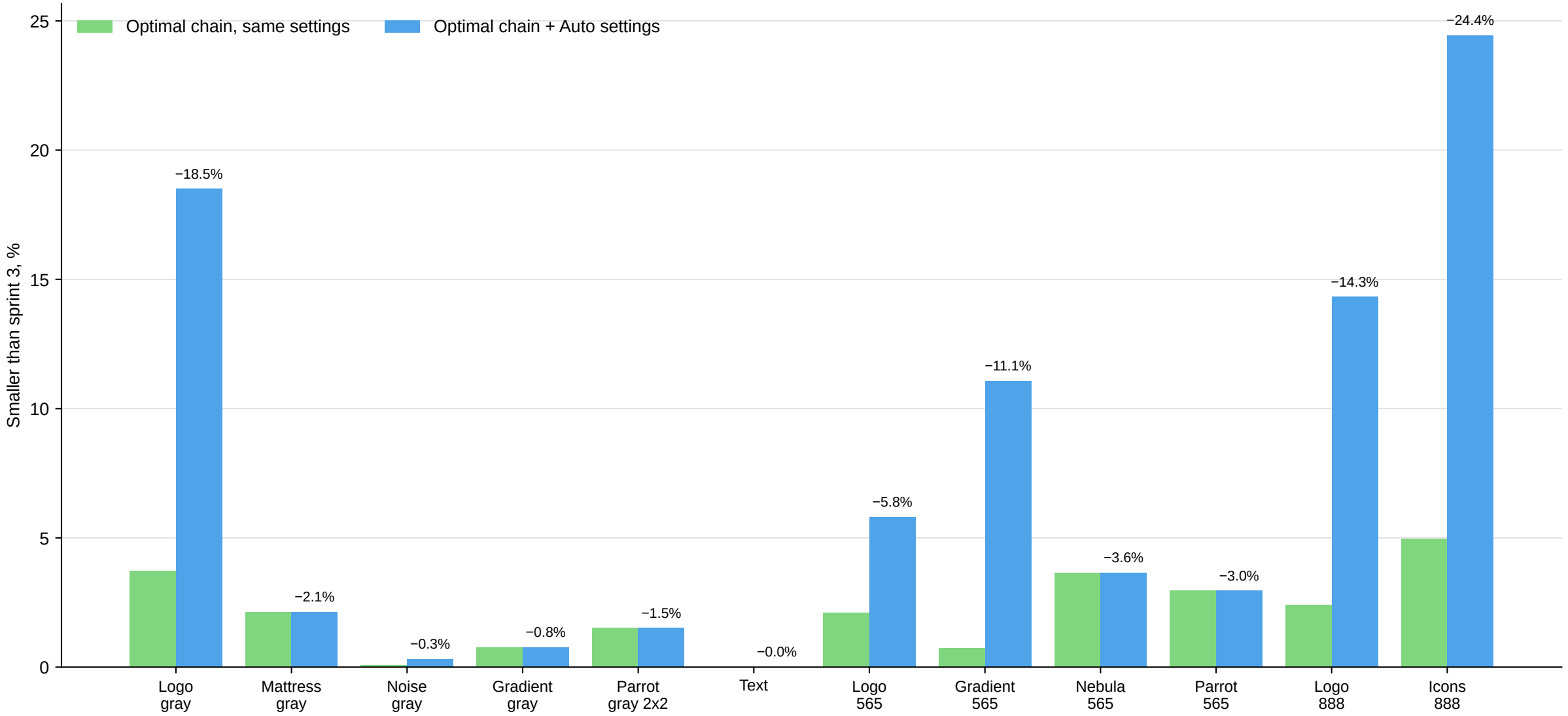
Auto may choose any Δ option. The decoder must be built with the matching #define in Decompression.h (DECOMPRESSION_FIXED_WINDOW_FIRST etc.).

Results



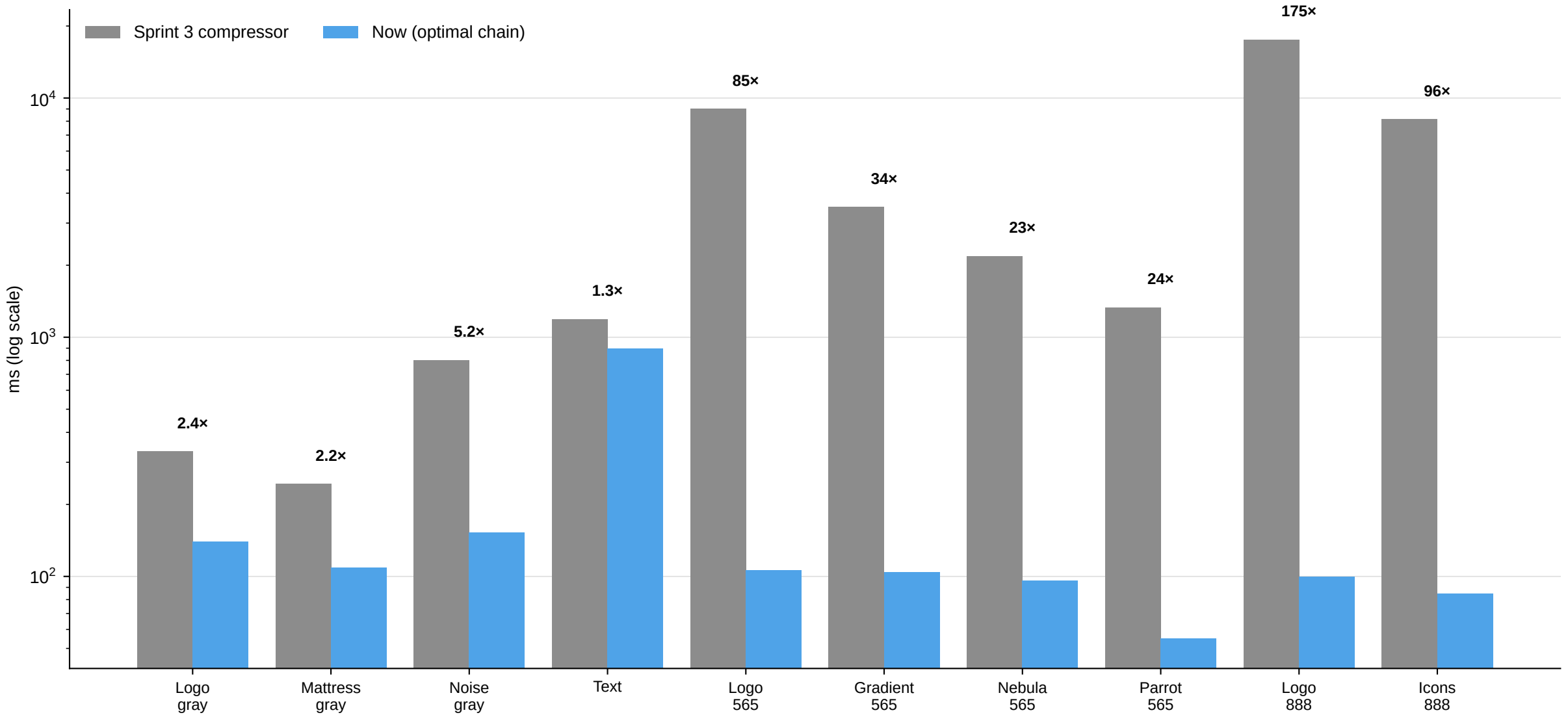
Sprint 3 settings for the first two series; 240 × 240 images, text 32 KB. Every output is checked with the C++ decoder.

Results: size reduction



Auto: block size field, Δ option and squares chosen per image (1–13 s per image on 4 cores).

Results: compression time



Same settings, one image at a time, 4-core cloud machine, .NET 10.

Decompression

- The stream format is unchanged: the existing decoder reads every new file.
- All the improvements are on the compressor side; the decoder stays as small as before:
  - per channel it keeps only the method, the window and the last value;
  - one pass over the data, no tables, no buffers beyond one block header.
- The Δ option and the channel bit depths must match the decoder build (#define).
- Squares need DECOMPRESSION_USE_SQUARES and the same square width.

Next sprint

- Cross-platform interface (Avalonia) on top of the UI-free core library.
- Repository cleanup, build and test automation, a proper release.
- Format extensions worth measuring (would change the decoder):
  - a flag "same methods as the previous block" to shorten headers;
  - variable-size length field for long uniform blocks.